Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
import hashlib

def hash_to_range(input_string: str) -> int:
     hash_object = hashlib.sha256(input_string.encode())
     hash_int = int(hash_object.hexdigest(), 16)
     return (hash_int % 3) + 1
input_string = "smit"
result = hash_to_range(input_string)
print(result)
```

```
1
```

▶ Question 1

Starter Code for Question 1

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional, List

class TreeNode:
    def __init__(self, val: int = 0, left: 'Optional[TreeNode]' = None, right: 'Optional[TreeNode]' = None):
        self.val = val
        self.left = left
        self.right = right

def is_duplicate(root: TreeNode) -> int:
    # TODO
    pass
```

▶ Question 2

Starter Code for Question 2

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional, List

class TreeNode:
    def __init__(self, val: int = 0, left: 'Optional[TreeNode]' = None, right: 'Optional[TreeNode]' = None):
        self.val = val
        self.left = left
        self.right = right

def bt_path(root: TreeNode) -> List[List[int]]:
    # TODO
    pass
```

▶ Question 3

Starter Code for Question 3

```
from typing import List

def missing_num(nums: List[int]) -> int:
    # TODO
    pass
```

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
The problem asks us to scan a list of integers from left to right and identify the first value that appears more than once. “Fi
```

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

Double-click (or enter) to edit

```
**New Example**
Input: "{[()()]}"
Output: True

**Explanation:**
Every opening bracket has a matching closing bracket of the same type, and all pairs close in the correct LIFO order. The stack
```

italicized text

- Copy the solution your partner wrote.

```
# If Q1 (example structure)
# from typing import List
# def first_duplicate(nums: List[int]) -> int:
#     seen = set()
#     for x in nums:
#         if x in seen:
#             return x
#         seen.add(x)
#     return -1

# If Q2 (example structure)
# def is_valid_brackets(s: str) -> bool:
```

```
#     match = {')': '(', ']': '[', '}': '{'}
#     stack = []
#     for ch in s:
#         if ch in '([{':
#             stack.append(ch)
#         else:
#             if not stack or stack[-1] != match.get(ch):
#                 return False
#             stack.pop()
#     return len(stack) == 0

# If Q3 (example structure)
# from typing import List
# def move_zeros_to_end(nums: List[int]) -> List[int]:
#     write = 0
#     for read in range(len(nums)):
#         if nums[read] != 0:
#             nums[write] = nums[read]
#             write += 1
#     for i in range(write, len(nums)):
#         nums[i] = 0
#     return nums
```

link text

- Explain why their solution works in your own words.

```
Their solution works because it scans the list left-to-right while keeping a set of values that have already appeared. For each
- If it’s not in the set, we add it and keep going.
- If it’s already in the set, that means we’ve just encountered the first time any number repeats during this traversal, so ret

This logic is correct because the scan order enforces what “first” means: the earliest moment a repeated value is detected. Usi
```

1. List item
2. List item

- Explain the problem’s time and space complexity in your own words.

```
he time complexity of their solution is **O(n)** because the function scans through the list once from left to right, and each

The space complexity is **O(n)** in the worst case. This happens when no number repeats, meaning every element gets stored in t
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
Their solution is based on the classic stack approach, which is the correct and optimal method for validating bracket sequences.

One improvement would be to add a short docstring or comments to clarify the purpose of the mapping dictionary and how the stack

Another minor suggestion is to explicitly check whether a character outside of `()[]{}` appears in the input, if the assignment

Overall, the solution is efficient, correct, and well-structured, with only small stylistic or documentation improvements possib
```

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
Completing Assignment 1 helped me strengthen my understanding of how to connect each problem to an appropriate data structure or
Presenting and reviewing my partner’s Assignment 1 provided a valuable opportunity to practice technical communication. Explainin
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- [ ] Created a branch with the correct naming convention.
- [ ] Ensured that the repository is public.
- [ ] Reviewed the PR description guidelines and adhered to them.
- [ ] Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

```
Start coding or generate with AI.
```